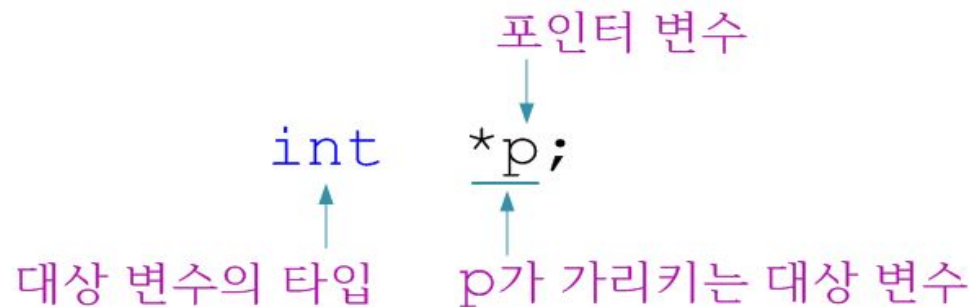


포인터

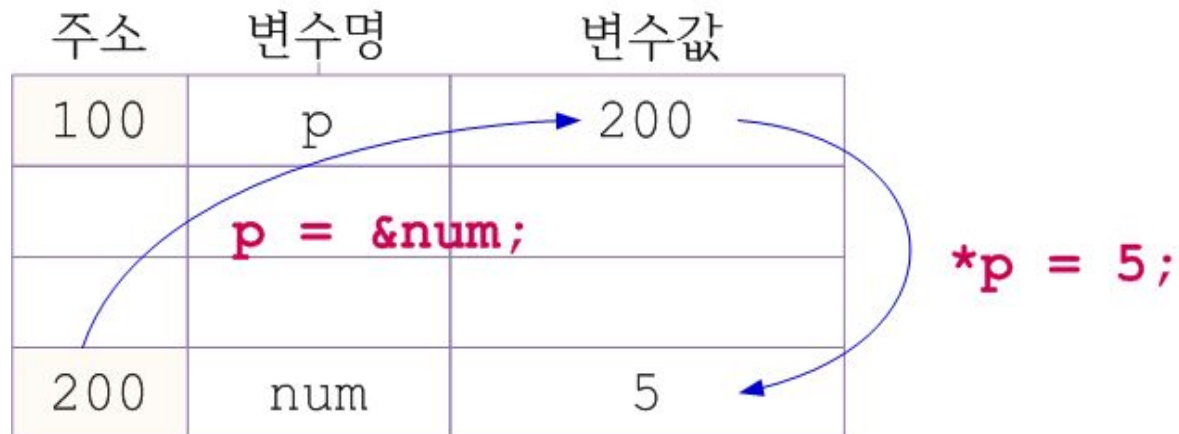
- 포인터 (Pointer) 또는 포인터 변수 (Pointer Variable)
 - 다른 변수를 가리키는 변수
 - 다른 변수의 주소를 담는 변수



- `*p`는 `p`가 가리키는 것을 의미
- Something pointed by `p` is integer type
- 변수 `p`가 어떤 주소 값을 담고 있는데 그 주소를 따라가면 다른 변수가 있고 그 변수는 타입이 정수다.
- 일반적인 변수 값을 담는 그릇이 아니라 주소 값을 담는 그릇

포인터 활용

```
int num;  
int *p;  
p = &num;  
*p = 5;
```



- **간접 참조** 또는 **참조** (Dereferencing, Indirection)
 - *p = 5; 이면 num에 **간접 접근**
 - num = 5; 이면 num에 **직접 접근**

SYMTAB(Symbol Table)

```
char *p;  
int *p;
```

- 포인터 선언 시 **대상의 타입을 명시**. WHY? 포인터가 가리키는 것은 변수의 **시작 주소**. 거기서부터 몇 바이트를 읽을지를 알아야 함.

실습 및 해설

SYMTAB			Main Memory
타입	주소	변수 명	
int	0028F718	num	5
...	...	...	...
pointer	0028F70C	p	0028F718

- SYMTAB에는 변수 타입, 주소, 변수 명이 저장되어 있음
 - 실행 시 이 도표를 참고로 변수에 접근 (Ex. *p => num)

어셈블리 언어와 포인터

OP code	Operand
---------	---------

- OP(eration) code(연산자), Operand(피 연산자)

```
LOAD AX, #200
```

```
a = 200;
```

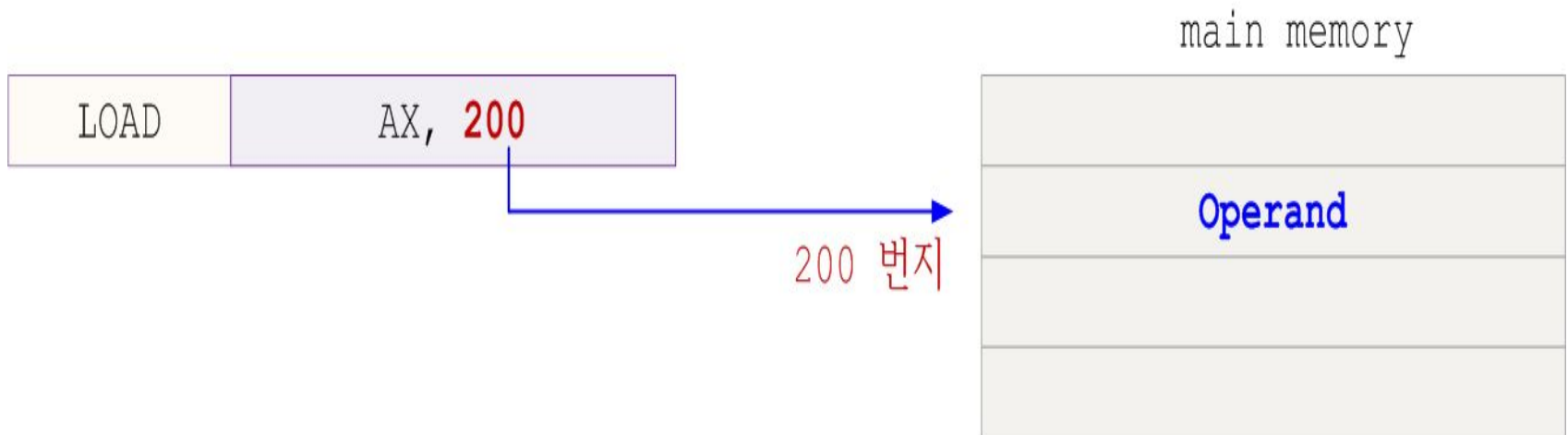
LOAD	AX, #200
------	----------

- 즉시 접근(Immediate Addressing)
 - 상수 200을 명령문에 저장
 - 메모리 접근 불필요

어셈블리 언어와 포인터

```
LOAD AX, 200
```

```
a = b;
```

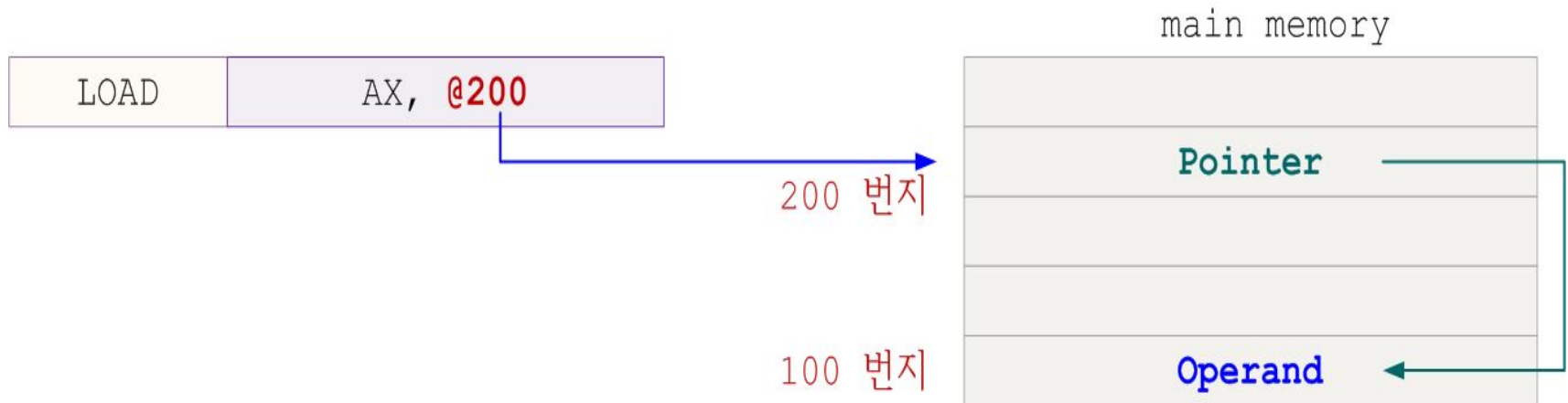


- 직접 접근 (Direct Addressing)
 - 피 연산자의 메모리 주소를 지정
 - SYMTAB에 변수 b의 주소가 200번지라면 이는 `a = b;`에 해당
 - 한 번의 메모리 접근

어셈블리 언어와 포인터

```
LOAD AX, @200
```

```
a = *b;
```



- 간접 접근 (Indirect Addressing)
 - 피 연산자의 메모리 주소를 담은 변수의 주소를 지정
 - C 언어의 포인터에 해당
 - 두 번의 메모리 접근
 - 즉시 접근, 직접 접근, 간접 접근 순으로 속도 저하
 - 접근할 수 있는 주소의 범위를 늘리는 수단
 - 포인터 변수의 비트 수 > 피 연산자의 비트 수

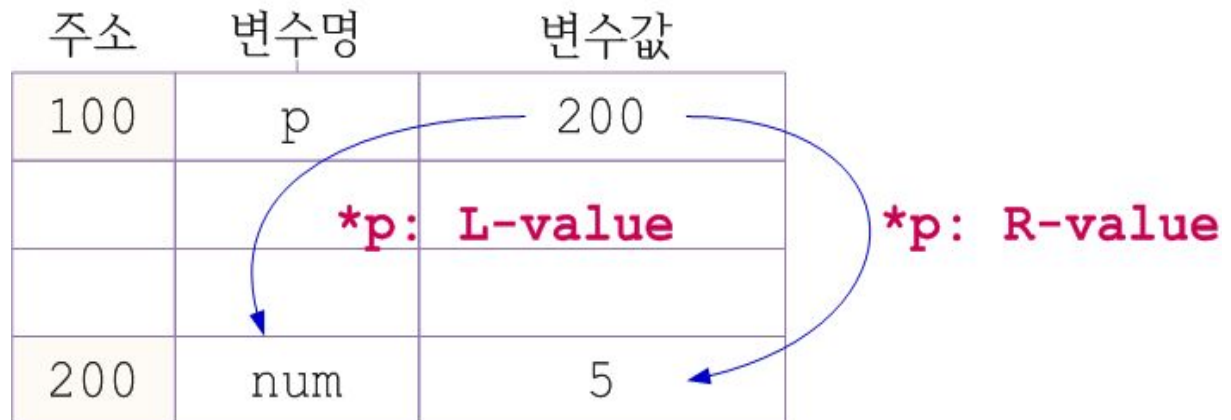
포인터 유의사항

```
int * p;    // o.k.  
int *p;     // 상동. p가 가리키는 것이 정수  
int* p;     // 상동. p는 정수 포인터  
int* p, q;  // q는 포인터가 아님  
            // int *p, *q; 라고 해야 함
```

```
int* p;  
p = 240;    // 우변이 (int*)240이 아니라서 경고                // 그러나  
직접 번지수를 대입할 경우 접근 위반  
            // 메모리 주소는 운영체제가 관리  
printf("Pointer value is %p.\n", p);
```

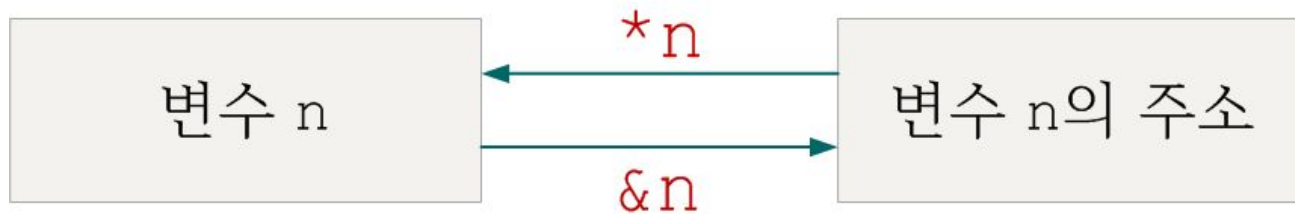
L-value, R-value

```
*p = 5;  
*p = *p + 5;    // 좌변은 L-value, 우변은 R-value  
printf("%d", *p); // 인자는 R-value  
&num = 5;       // 오류. 주소 연산의 결과는 상수로서 R-value
```



- *p가 L-value
 - 변수 자체를 의미
- *p가 R-value
 - 변수의 값을 의미

● 실습 및 해설



- 역함수 (Inverse Function)
 - 곱하기와 나누기는 역함수 관계
 - $*n$ 과 $\&n$ 은 역함수 관계
 - $\&$ 는 주소 연산자로서 변수로부터 주소를 찾으라는 것
 - $*$ 는 참조 연산자로서 주소로부터 변수를 찾으라는 것
 - 역함수끼리는 상쇄

$*(\&n)$ is equal to n .

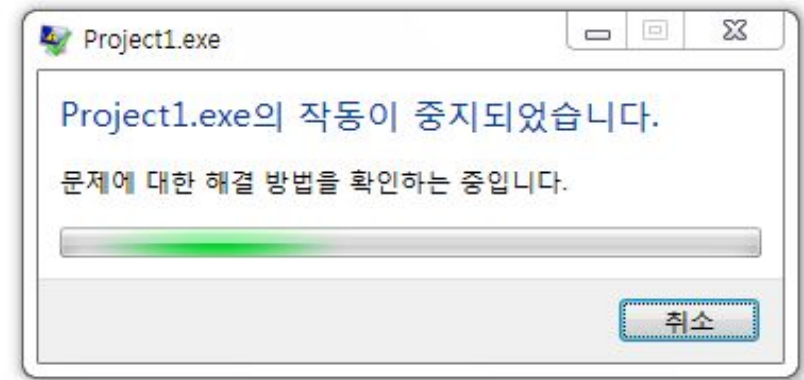
$*(*(\&n))$ is equal to $*n$.

포인터 관리

```
double num = 2.0;  
int *p;  
p = &num;
```

- Type Mismatch Error

```
int *p = NULL;  
*p = 100;      // Error
```



- NULL은 가리키는 대상이 없다는 의미
- *p = 100;에 의해 프로그램 실행 중 크래쉬 창이 뜸
- C 언어에서 포인터 관리는 프로그래머의 책임
 - 비롯한 대부분 언어가 포인터를 제공하지 않음 (WHY?)
 - 포인터는 C 언어의 단점이자 최대 장점

NULL 포인터

```
int *p = NULL;
#define NULL 0
#define NULL ((void *) 0)
```

- 매크로 상수 NULL
 - 0 번지가 아니라 아무 것도 가리키지 않는다는 의미

```
int *p = NULL;
...
if (p == NULL) {
    printf("p points to nothing.\n");
    exit(1);
}
*p = 100;
```

- 가리키는 것이 없는 포인터에는 NULL을 대입 (초기화 포함)
- 포인터를 따라갈 때에는 미리 NULL인지 확인

타입 지정자 const

```
int num = 1;
const int *p = &num;    // *p는 상수
*p = 5;                 // 오류
```

```
int num1 = 1, num2 = 2;
int* const p = &num1;    // p는 상수
*p = 5;                  // O.K.
p = &num2;                // 오류
```

```
const int num = 10;      // num은 상수
const int *p = &num;      // O.K, *p는 상수
int *p = &num;            // 컴파일러 경고 (Warning)
                        // *p(num)를 바꿀 우려
```

포인터에 의한 참조 호출

● 실습 및 해설

	변수	주소	변수값
call_by_value	p		500 → 501
			copy
main	num	200	500

	변수	주소	변수값
call_by_reference	p		200
main	num	200	500 → 501

**p += 1;*

&num

copy

200

● 참조 호출

- 변수 값(500)이 아니라 변수의 주소 값(200번지)을 복사
- 복사된 값을 따라가서 원본 변수 값을 변경
- C 언어는 주소를 통해서는 아래에 깔린 스택 프레임에 접근 허용

● 실습 및 해설

- 큰 수와 작은 수 구하기
- `void min_max(int a, int b, int* min, int* max);`
- a, b는 **입력 인자**
 - 값 호출
 - 대개 읽기 전용. 쓰기를 하더라도 원본은 불변
- min, max는 **출력 인자**
 - 참조 호출
 - 함수 내부에서 min, max를 변경하면 원본 변경
 - 함수의 리턴 값이 두 개인 셈

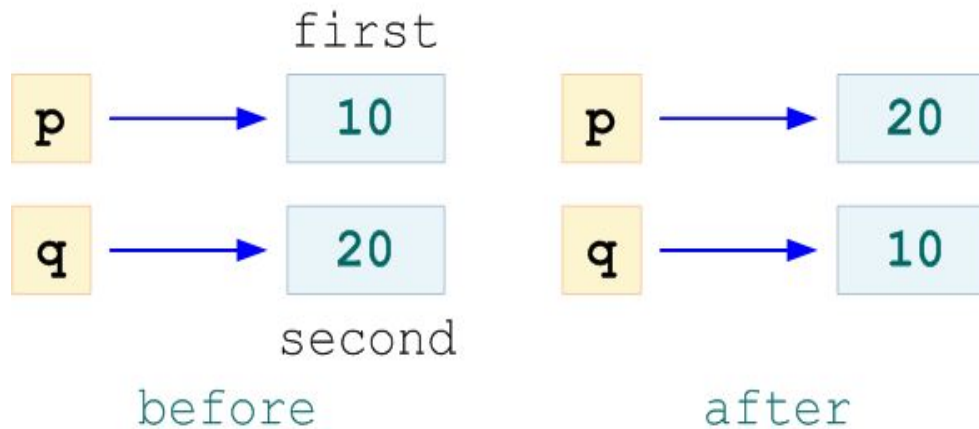
포인터에 의한 원본 스왑

● 실습 및 해설

- `int* larger(int* a, int* b);` // 포인터를 리턴하는 함수

● 실습 및 해설

- `swap(&first, &second);`
- ```
void swap(int *p, int *q) {
 int temp;
 temp = *p;
 *p = *q;
 *q = temp;
}
```



## 배열 단위의 대입 및 복사

```
int a[4] = {1, 2, 3, 4};
int b[4] = {5, 6, 7, 8};
a = b; // Error
if (a == b) // Error
for (i = 0; i < 4; i++) // O.K.
 a[i] = b[i];
```

- C 언어는 배열 단위의 대입이나 복사를 허용하지 않는다.
  - WHY? C 언어 설계 과정에서 실행 속도를 고려
- 배열명은 그 자체로 배열의 시작 주소
  - 배열명은 상수 포인터
    - a, b는 배열이 만들어질 때 할당된 시작 주소(불변)
    - 상수는 L-value가 아니기 때문에 좌변에 올 수 없음
  - (a == b)
    - 배열 요소가 같은지를 비교하는 것이 아님
    - 배열의 시작 주소가 같은지를 비교. 당연히 다름



## 배열을 인자로 전달하기

### ● 실습 및 해설

- `int rabbit[MAX] = {10, 20, 30, 40};` // 배열 선언
- `increment(rabbit);` // 함수 호출
- `void increment(int rb[ ]);` // 피 호출 함수

- 배열 명 = 배열 시작 주소 = 포인터
- 배열 명을 인자로 전달하면 포인터가 복사되어 넘어감
  - 참조 호출 효과를 기할 수 있음

|                      | 변수 명      | 시작 주소 | 변수 값    |
|----------------------|-----------|-------|---------|
| increment의<br>스택 프레임 |           |       |         |
|                      | rb        | 96    | 100     |
| main의<br>스택 프레임      | rabbit[0] | 100   | 10 → 11 |
|                      | rabbit[1] | 104   | 20 → 21 |
|                      | rabbit[2] | 108   | 30 → 31 |
|                      | rabbit[3] | 112   | 40 → 41 |

## 배열 인자 선언

- `void increment(int rb[ ]);`
  - `rb`는 배열 명이자 포인터
- `void increment(int rb[MAX]);`
  - 상동.
  - 컴파일러는 포인터 값만 복사. 요소 개수에는 무관심
- `Void increment(int* rb);`
  - 상동.
  - 포인터 `rb`가 가리키는 것이 `int`. 배열이라는 의미가 약함

```
increment(rabbit[3]); // 배열 요소 전달
void increment(int rb3); // 값 호출
```

```
void increment(const int rb[])
 // 원본 배열을 읽기만 할 때에는 const를 붙이는 것이 좋음
```